

**Yenepoya Institute of Arts, Science,
Commerce, and Management**

Project – High Level Design

on

**Cloud Forensics Automation for
E-commerce Breaches**

Student Name: Ardra P V

**Industry
Mentor:**

Mr. Shashank

Guided by: Ms. Suzana
Sharol Prabhakar

Table of Contents

No.	Section Title	Page
1	Introduction	
1.1	Scope of the Document	
1.2	Intended Audience	
1.3	System Overview	
2	System Design	
2.1	Application Design	
2.2	Process Flow	
2.3	Information Flow	
2.4	Components Design	
2.5	Key Design Considerations	
2.6	API Catalogue	
3	Data Design	
3.1	Data Model	
3.2	Data Access Mechanism	
3..3	Data Retention Policies	
3.4	Data Migration	
4	Interfaces	
5	State and Session Management	
6	Caching	
7	Non-Functional Requirements	
7.1	Security Aspects	
7.2	Performance Aspects	
8	References	

1. Introduction

The Cloud Forensics Automation System is designed to automate the process of analyzing file metadata to detect potential tampering or suspicious modifications. This High-Level Design (HLD) document provides an overview of system architecture, major components, data flow, and technology choices.

1.1 Scope of the Document

In Scope:

- Overall system architecture
- High-level component interactions
- Data flow across modules
- Technology stack

Out of Scope:

- Function-level implementation
- Detailed algorithms
- Code-level logic

1.2 Intended Audience

- System Architects – for architecture validation
- Developers – for understanding system structure
- Evaluators – for design assessment

1.3 System Overview

The system allows users to upload files, analyze metadata, detect suspicious modifications, and visualize results through a dashboard. It operates as a lightweight, single-user application using a Flask-based backend and local storage.

2. System Design

2.1 Application Design

The system follows a **modular monolithic architecture**, where all components are part of a single application but logically separated into modules:

- User Interface Layer
- Processing Layer
- Storage Layer

2.2 Process Flow

1. User uploads file(s)
2. Backend saves file securely
3. Metadata is extracted
4. Detection logic is applied
5. Results stored in CSV
6. Dashboard displays results

2.3 Information Flow

Input:

- Uploaded files

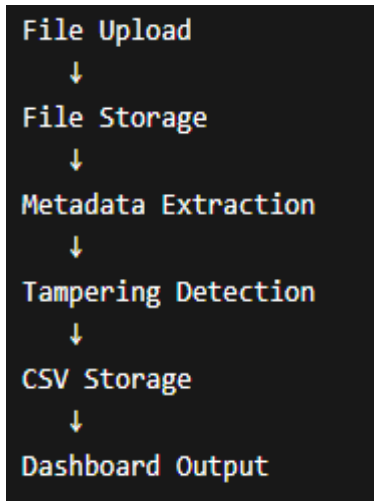
Processing:

- Extract timestamps (Creation time ctime, modified time - mtime)
- Calculate file size
- Apply tampering logic

Output:

- CSV report
- Dashboard table with suspicious flags

Figure: Information Flow of File Analysis System



2.4 Components Design

1. File Handling Module

- Handles file upload and storage
- Uses secure filename sanitization

2. Metadata Extraction Module

- Extracts:
 - Creation time
 - Modified time
 - File size

3. Detection Engine

- Compares timestamps
- Flags suspicious files

4. CSV Manager

- Writes structured results
- Reads data for dashboard

5. Dashboard Module

- Displays results
- Highlights suspicious entries

2.5 Key Design Considerations

- ☐ Simplicity over scalability
- ☐ Local storage for privacy
- ☐ Minimal external dependencies (Flask only)
- ☐ Fast processing for small datasets

2.6 API Catalog

Endpoint	Method	Description	Request	Response
/upload	POST	Upload and analyze files	File	Success / Error
/dashboard	GET	Display results	-	Data list
/clear	POST	Delete data	-	Success
/download-report	GET	Download CSV	-	CSV file

3. Data Design

3.1 Data Model

Field Name	Type	Description
-----	-----	-----
filename	string	Name of file
created_time	datetime	File creation time
modified_time	datetime	Last modified time
file_size	int	Size of file in bytes
suspicious_flag	boolean	Indicates tampering

3.2 Data Access Mechanism

- Uses file-based storage (CSV)
- Read/Write using Python csv module
- No ORM used

3.3 Data Retention Policies

- Data stored locally in CSV file
- Retained until manually deleted by user
- No automatic cleanup mechanism

3.4 Data Migration

- Not applicable (no database used)

4. Interfaces

- External Interfaces

- None (fully local system)

Internal Interfaces

Caller	Callee	Purpose
Upload Route	File Handler	Save file
File Handler	Metadata Extractor	Extract data
Extractor	Detection Engine	Check tampering
Detection Engine	CSV Manager	Store result
Dashboard	CSV Manager	Fetch data

User Interface

- Upload page
- Dashboard page

Configuration Interface

- Flask config variables:
 - UPLOAD_FOLDER
 - MAX_CONTENT_LENGTH

5. State and Session Management

Component	State	Storage
Upload Process	Temporary files	File system
Results	Analysis data	CSV
Session	Flash messages	Flask session

6. Caching

- No caching implemented
- System processes data in real-time without reuse of previous results

7. Non-Functional Requirements

7.1 Security Aspects

- Secure filename handling (prevents path traversal attacks)
- File size limit (50MB) to prevent system overload
- No external exposure (system runs locally)

7.2 Performance Aspects

- Fast execution using local file processing
- Handles small datasets efficiently (low latency)
- Minimal resource usage (CPU and memory)
- No dependency on external APIs or network call

8. References

1. **Flask Documentation**

Official documentation for Flask framework used in backend development

<https://flask.palletsprojects.com/>

2. **Python os Module Documentation**

Used for file metadata extraction (ctime, mtime, file size)

<https://docs.python.org/3/library/os.html>